

GenAi Training Day 4

Chaining with LangChain

Overview

- Real AI apps need more than single LLM calls
- Complex tasks require multiple steps, LLM calls, memory, and tools
- One LLM call = one response
- Manual prompt chaining becomes messy and unreliable
- LangChain turns prompts into structured AI systems

What Is LangChain?

- Framework for building LLM-powered applications
- Connects LLMs with data, tools, and memory
- Orchestrates multi-step workflows cleanly

Why LangChain Exists?

- Tasks require step-by-step reasoning
- Output of one step feeds the next
- Enables scalable, maintainable pipelines

Key Components

- LLMs (model abstraction)
- Prompts
- Chains (workflow logic)
- Retrievers (context fetching)
- Memory (state management)
- Tools (external actions)

LangChain vs Raw LLM Calls

- **Raw LLM:** one prompt = one response
- **LangChain:** multiple steps = final output
- Swap LLM providers without changing logic

Sequential Chains

- Multiple chains executed in order
- Output of one step becomes input to the next
- Ideal for linear workflows (summarize → extract → generate)

Memory in LangChain

- Stores conversation history and context
- Enables continuity across interactions
- **Types:** buffer, summary, vector-based
- Memory manages context but not learning

Tools & Actions

- Allow LLMs to call APIs and scripts

- Enables live data access and automation
- Moves LLMs from text to action

Multi-Step Chains

- Break complex tasks into clear steps
- Single-purpose chains improve reliability
- Combined workflows reduce errors

Why It Matters?

- Enables production-ready GenAI apps
- Promotes modular, maintainable design
- Scales from prototypes to real systems